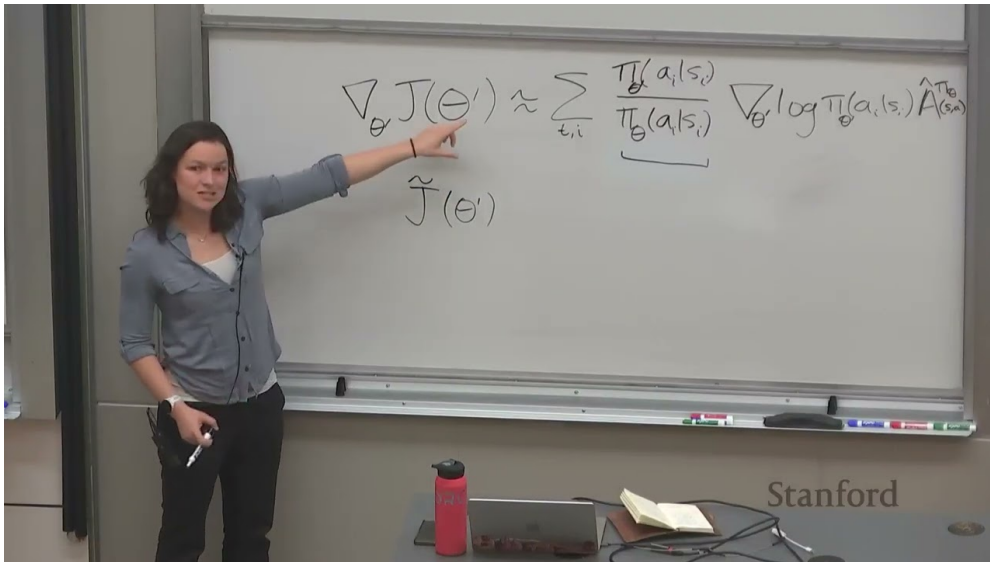


课程笔记

CS224R Lecture 5: Off-Policy Actor-Critic

基于公开课程资料整理

2025 年春季



视频作者/频道: Stanford Online

发布日期: 2025-04

视频时长: 约 80 分钟

讲者: Chelsea Finn

课程主页: cs224r.stanford.edu

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 多步梯度更新的问题	3
1.1 Importance Weights 回顾	3
1.2 本章小结	3
2 训练监控与日志体系	3
2.1 日志的结构化写法	3
2.2 指标面板	3
2.3 本章小结	3
3 Actor-Critic 数据流水线	4
3.1 采集、存储与分发	4
3.2 优势估计与归因分配	4
3.3 本章小结	4
4 PPO: Proximal Policy Optimization	4
4.1 核心思想：限制策略偏移	4
4.2 PPO 完整算法	5
4.3 本章小结	5
5 SAC: Soft Actor-Critic	5
5.1 更彻底的 Off-Policy	5
5.2 拟合 Q 函数	5
5.3 策略更新	5
5.4 SAC 完整算法	6
5.5 本章小结	6
6 Replay Buffer 管理与数据挑选	6
6.1 为何大小与采样策略关键	6
6.2 清洗回放数据：过滤 vs. 补充	6
6.3 本章小结	7
7 超参调优与实验复现	7
7.1 常见超参数组合	7
7.2 复现要点	7
7.3 本章小结	7
8 策略安全与正则化	8
8.1 防止策略崩溃的 Regularization	8
8.2 监控策略行为	8
8.3 本章小结	8

9 Reward Shaping 与价值稳定	8
9.1 价值函数稳定的微调技巧	8
9.2 Shaping 的设计原则	8
9.3 本章小结	9
10 评估与部署建议	9
10.1 多维评估池	9
10.2 部署图谱	9
10.3 本章小结	9
11 幻灯片精华	10
11.1 核心概念图	10
11.2 PPO vs SAC 结构对比	10
11.3 Q 函数稳定性案例	11
11.4 Replay Buffer 与 Sim2Real	11
11.5 策略蒸馏流程图	12
11.6 性能雷达	12
11.7 模拟-真实迁移示意	13
11.8 训练与部署工作流	13
11.9 调试与日志示例	14
11.10 Overfit 与 Regularization 诊断	14
11.11 使用 entropy 进行监控	15
11.12 部署后的长期监控	15
11.13 本章小结	15
12 案例对比与设计模式	16
12.1 PPO / SAC 在不同任务上的表现	16
12.2 工程设计模式	16
12.3 本章小结	16
13 Sim2Real 与策略蒸馏	16
13.1 Sim2Real 的分布转移策略	16
13.2 策略蒸馏与小模型部署	17
13.3 本章小结	17
14 总结与延伸	17
14.1 拓展阅读	17
14.2 总结表与工程路线	18
15 附录：常见实践问题	18

1 多步梯度更新的问题

On-policy actor-critic 每收集一批数据只能做一步梯度更新，效率很低。我们希望在同一批数据上做**多步更新**。

1.1 Importance Weights 回顾

用旧策略 π_θ 的数据来更新新策略 $\pi_{\theta'}$ 时，需要 importance weights：

$$\nabla_{\theta'} J(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_\theta(a_{i,t} | s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} | s_{i,t}) \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t})$$

多步更新的风险

如果对代理目标函数（surrogate objective）做很多步梯度上升，策略会被激励大幅偏离旧策略。此时 importance weights 不再可靠，优势估计也失效，导致过拟合和性能崩塌。

1.2 本章小结

多步梯度更新能提高数据效率，但需要约束策略更新的幅度来保持稳定性。

2 训练监控与日志体系

2.1 日志的结构化写法

复制整个训练过程的关键是详细的 logging schema。以下字段是最低要求：

- 'step', 'episode', 'wall_time', 'slice', 'avg_return', 'std_return', 'kl_divergence', 'entropy', 'policy'
- 'buffer_size', 'td_error_mean', 'td_error_std', 'Replaybuffer', 'learning_rate', 'grad_norm', 'value_loss', 'valuehead'

日志粒度与报警

高频 logging（每 1k step）结合 threshold-based alarms（KL > 0.06, return variance > 0.2）能在 divergence 发生前打断训练，特别是在 Sim2Real 环境中避免一次物理损坏。

2.2 指标面板

实时可视化 dashboard 通常至少包含：

1. PPO ratio distribution histogram
2. SAC entropy Q-value curves
3. Batch-wise advantage histogram to detect saturation

2.3 本章小结

结构化 logging 与实时 dashboard 能让研究者及时发现 divergence、explosion、overfitting 等 failure modes，从而更快复现核心表现。

3 Actor-Critic 数据流水线

3.1 采集、存储与分发

Actor-Critic 框架的数据流可以拆成四个阶段：采集 (rollout)、存储 (buffer)、处理 (advantage estimation)、分发 (learning step)。每个阶段都有陷阱。例如，采集阶段如果数据全部来自一条 deterministic policy，会产生 low diversity，导致 replay buffer 中出现大量 near-duplicate transitions，降低实际更新的有效样本数。

Rollout 与 buffer 的协同

为了保持采集多样性，通常采用 epsilon/entropy noise、随机策略混合等技巧；同时尽量避免把同一个轨迹重复写入 buffer，至少在 enqueue 时做 tenure check（记录 transition 被访问次数）。

3.2 优势估计与归因分配

优势估计 (advantage estimation) 是 Actor-Critic 的中心。常用的算法为 GAE(λ)，它在 bias-variance 之间做 trade-off：

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

选择 $\lambda = 0.95$ 是常见的默认值，同时需要保持值函数 $\hat{V}$ 的训练稳定，否则 advantage 的方差会暴涨。

3.3 本章小结

Actor-Critic 的数据流水线覆盖 rollout、buffer、advantage estimation、update step。每个环节都需要监控，以防剧烈偏移带来的 instability。

4 PPO: Proximal Policy Optimization

4.1 核心思想：限制策略偏移

两种策略：

方法 1：KL 约束

$$\max_{\theta'} \tilde{J}(\theta') \quad \text{s.t.} \quad \mathbb{E}_{s \sim \pi_\theta} [D_{\text{KL}}(\pi_{\theta'}(\cdot | s) \| \pi_\theta(\cdot | s))] \leq \epsilon$$

在 LLM 的偏好优化中非常常见。

方法 2：Clipping

不直接约束策略，而是裁剪 importance weights，移除策略大幅偏离的激励：

$$\tilde{J}(\theta') = \sum_{t,i} \min \left(\frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_\theta(a_{i,t} | s_{i,t})} \hat{A}^{\pi_\theta}, \text{clip} \left(\frac{\pi_{\theta'}(a_{i,t} | s_{i,t})}{\pi_\theta(a_{i,t} | s_{i,t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}^{\pi_\theta} \right)$$

PPO 的核心目标

这就是 PPO 的最终 surrogate objective。通过 clipping 和 min 操作，策略不再有激励大幅偏离旧策略，即使做多步更新也能保持稳定。

4.2 PPO 完整算法

1. 从当前策略 π_θ 采集一批数据
2. 拟合价值函数 $\hat{V}_\phi^\pi$
3. 计算优势 $\hat{A}^{\pi_\theta}$
4. 在这批数据上对 PPO objective 做多步梯度更新（约 3-10 步）
5. 重复

4.3 本章小结

PPO 通过 clipping importance weights 来限制策略偏移，允许在同一批数据上做多步更新，是目前最广泛使用的 on-policy RL 算法之一。

5 SAC: Soft Actor-Critic

5.1 更彻底的 Off-Policy

PPO 仍然需要定期重新收集数据。SAC 走得更远：使用 **replay buffer** 存储所有历史数据，从中随机采样来更新 Q 函数和策略。

5.2 拟合 Q 函数

SAC 直接拟合 Q^π 而不是 V^π ，使用 TD 更新：

$$\min_{\phi} \sum_{(s,a,s') \in \mathcal{D}} \left\| \hat{Q}_{\phi}^{\pi}(s,a) - \left(r(s,a) + \gamma \mathbb{E}_{a' \sim \pi_{\theta}(\cdot|s')} \left[\hat{Q}_{\phi}^{\pi}(s',a') \right] \right) \right\|^2$$

为什么 Q 函数可以用 off-policy 数据

Q 函数的 TD target $r(s,a) + \gamma \mathbb{E}_{a' \sim \pi} [Q(s',a')]$ 中， a' 是从**当前策略**采样的（而不是从数据中读取的）。因此 (s,a,s') 转移数据可以来自任何策略，只要 a' 是新采样的即可。

5.3 策略更新

策略优化目标加入了**熵正则化**：

$$\max_{\theta} \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta}(\cdot|s)} \left[\hat{Q}^{\pi}(s,a) - \alpha \log \pi_{\theta}(a | s) \right]$$

熵项 $-\alpha \log \pi_{\theta}$ 鼓励策略保持一定的随机性，有助于探索。

5.4 SAC 完整算法

1. 用当前策略与环境交互，将转移 (s, a, r, s') 存入 replay buffer $\mathcal{B}$
2. 从 $\mathcal{B}$ 采样 mini-batch
3. 更新 Q 函数（用 TD target）
4. 更新策略（最大化 Q + 熵）
5. 重复步骤 1-4

PPO vs. SAC

- **PPO**: On-policy（加少量 off-policy），用 clipped surrogate objective。每次收集新数据后做几步更新。更稳定、更易调参。
- **SAC**: 完全 off-policy，用 replay buffer + Q 函数。数据效率更高。适合连续动作空间。

两者都是目前最常用的深度 RL 算法。

5.5 本章小结

SAC 通过拟合 Q 函数并使用 replay buffer 实现了完全 off-policy 的 actor-critic 算法，数据效率远高于 on-policy 方法。熵正则化鼓励探索并提高稳定性。

6 Replay Buffer 管理与数据挑选

6.1 为何大小与采样策略关键

Replay buffer 提供多轮训练的数据，但它的设计直接影响性能。两个核心维度是容量和采样策略：

- **容量**：太小会频繁覆盖旧样本而丢失多样性，太大则降低训练时效。实务中通常选择能存下数百万条 transition。
- **采样策略**：均匀采样最简单，Prioritized Experience Replay 允许高 TD error 的 transition 更常被访问。

采样策略对效果的直接影响

High TD 文本的 transition 通常包含更难的决策或崩盘场景，优先返回它们可以让 Q 函数更快收敛；但如果优先级过度，可能造成估计偏差。通常的折衷是使用 β 衰减为 1 的 importance-sampling 权重来校正偏置。

6.2 清洗回放数据：过滤 vs. 补充

每当 agent 执行新策略，就应评估新采集数据的质量：

- **过滤**：移除 reward 异常、超出边界的 transition（例如撞墙后得 reward 为 1），避免训练噪音；

- 补充：人为合成带有特定标签的 transition（例如 reward shaping、domain randomization）帮助 agent 学习罕见但关键的情境。

回放 buffer 的污染效应

如果 buffer 长期积累低质量数据（例如 policy 崩溃后的 exit transitions），模型可能会在错误的示例上继续优化。定期清空 buffer 7% 左右最低 reward 的样本并重新收集，可以避免“data poisoning”。

6.3 本章小结

Replay buffer 的容量、采样策略和数据清洗策略共同决定 off-policy 学习的稳定性。适度的优先级采样、IS 校正以及对异常 transition 的剔除/采样，可以避免模型在旧数据上过拟合或被噪音拖累。

7 超参调优与实验复现

7.1 常见超参数组合

在 CS224R 的实验中，Chelsea 和团队使用以下组合取得了稳定表现：

- **PPO**: clip $\epsilon = 0.1$ 、batch size=64、learning rate=2e-4、value coeff=0.5、entropy coeff=0.01。
- **SAC**: Q learning rate=3e-4、policy learning rate=3e-4、replay buffer size=10e5、batch size=256、target smoothing coeff=0.005。
- **Replay buffer**: prioritized sampling weight $\alpha = 0.6$ 、importance sampling exponent β 从 0.4 线性增长到 1.0。

7.2 复现要点

对抗性地复现实验需要固定随机种子和 deterministic rollout。建议每次 run 记录

- seed、env name、max episode length
- optimizer state (Adam momenta) 与 learning rate schedule
- buffer size、prioritization hyperparameters

种子敏感性

为防止偶发崩溃，必须在 logging 中包括每条 rollout 的 return、length、advantages 的 std。若 return variance 超出预期，应立即停止训练，排查是否 replay buffer 中充斥着 high-loss transitions。

7.3 本章小结

超参数配比与记录日志是实验重现的关键。固定 seed、记录 optimizer state、监控 variance 并设置自动中断阈值，能显著降低 long-tail failure 的出现概率。

8 策略安全与正则化

8.1 防止策略崩溃的 Regularization

无论是 PPO 还是 SAC，当策略更新过猛都会导致性能崩盘。实务中常用的 regularization 包括：

- **KL penalty**：在 objective 中加入 $\lambda D_{\text{KL}}(\pi_{\theta'} \parallel \pi_{\theta})$ ，直接惩罚过大偏移。
- **Entropy decay**：在 SAC 中调节 α ，让策略在训练后期逐步减少探索以收敛。
- **Gradient clipping**：限制梯度范数避免步长过大。

8.2 监控策略行为

推荐的监控指标：

- **Average KL divergence** between successive policies
- **Episode variance** of returns—high variance signals instability
- **Q-value rollout test**—simulate fixed rollout to check escalation

自动告警设计

设定门限（例如 $\text{KL} > 0.05$ 时报错，episode std dev 增加 30

8.3 本章小结

安全调度需要 regularization 与监控指标结合。KL penalty、entropy decay、gradient clipping 是常见防过度偏移手段，而 KL、variance 和 Q-value rollout 监控提供实时风险预警。

9 Reward Shaping 与价值稳定

9.1 价值函数稳定的微调技巧

在 off-policy 更新中，Q 函数预测 drift 会严重影响 actor。实践中常见的 stabilizer 包括：

- **Target smoothing**：在 TD target 中使用 twin Q average + Polyak average smoothing coefficient τ 。
- **Reward clipping**：将 reward 限制在 $[-1, 1]$ ，避免爆炸性 gradient。
- **Value normalization**：在 batch level Q *scaled drift*

9.2 Shaping 的设计原则

Reward shaping 需要遵从 potential-based shaping theorem，确保 shaping term 不改变最优策略：

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s)$$

其中 Φ 是 potential function。一个实用的 $\Phi(s)$ 例子是 distance-to-goal, 提升 agent 在 early stages 的 guiding signal。

错误 shaping 的后果

若 shaping 直接奖励“接近目标”而非“完成任务”, agent 可能提前停留在潜在 reward 高的状态, 造成 reward hacking。这就是为什么 potential-based shaping 需严格依据 MDP structure。

9.3 本章小结

Reward shaping 和 value stabilization 是保持 long-term training 稳定的核心。target smoothing、reward clipping 与 potential-based shaping 搭配使用可以有效避免 value explosion 与 reward hacking。

10 评估与部署建议

10.1 多维评估池

训练完成后不要仅仅依赖单个 benchmark, 建议的 multi-evaluation pool 包括:

- **Policy rollouts** in multiple seeds and environments
- **Stress tests** with adversarial perturbations (e.g., shifted dynamics)
- **Human-in-the-loop scoring** for safety-critical behaviors

10.2 部署图谱

Deployment checklist:

1. Validate ability to recover from buffer corruption / reset seeds
2. Ensure logging of policy drift and extreme rewards
3. Provide rollback snapshot and auto-stop when variance spikes

Sim2Real 的守则

在现实部署中, 必须有 fallback policy (例如 script policy) 以及 safety net monitors; 同时 log policy decisions to enable post-hoc analysis—这一点对于医疗/航行应用尤为重要。

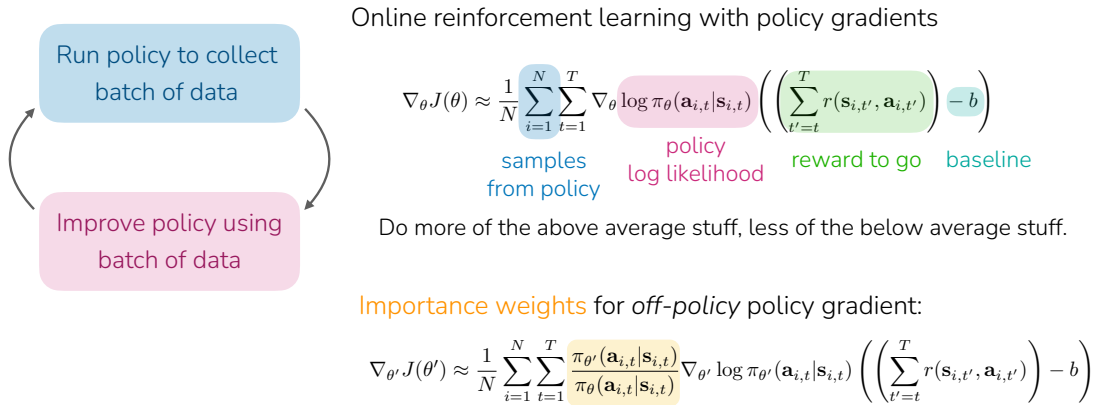
10.3 本章小结

评估应结合多样化 rollouts、adversarial stress tests 与人类打分。部署前建 checklist、snapshot 与 rollback 机制可避免生产环境的失控。

11 幻灯片精华

11.1 核心概念图

Recap: Policy Gradients



3

图 1: Lecture slide 重述 off-policy actor-critic 数据流与 surrogate objective。

11.2 PPO vs SAC 结构对比

Recap: Actor-Critic Methods

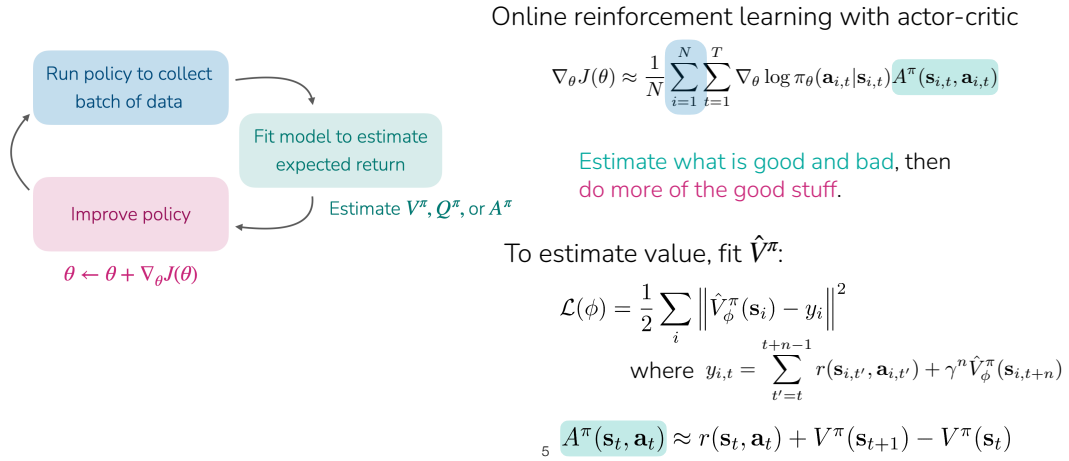


图 2: Slide 中对比图展示 PPO 的 clipping objective 与 SAC 的 entropy regularization。

11.3 Q 函数稳定性案例

The plan for today

Off-policy actor critic methods

1. Taking multiple gradient steps
 - a. Importance weights
 - b. Constraining step size with KL penalty or clipping
 - c. Practical PPO algorithm
- b. Even more off-policy algorithm
 - a. Fitting Q-value functions with data from other policies
 - b. Practical SAC algorithm

Key learning goals:

- All of the key concepts for practical algorithms like PPO and SAC

6

图 3: Slide 展示 Q 函数爆炸 vs smoothing 的实证曲线。

11.4 Replay Buffer 与 Sim2Real

Off-Policy Actor-Critic Methods

Version 1: Multiple Gradient Steps

1. Sample batch of data $\{(\mathbf{s}_{1,i}, \mathbf{a}_{1,i}, \dots, \mathbf{s}_{T,i}, \mathbf{a}_{T,i})\}$ from π_θ
2. Fit $\hat{V}_\phi^{\pi_\theta}$ to summed rewards in data
3. Evaluate $\hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) = r(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \gamma \hat{V}_\phi^{\pi_\theta}(\mathbf{s}_{t+1,i}) - \hat{V}_\phi^{\pi_\theta}(\mathbf{s}_{t,i}) \quad \forall t, i$
4. Evaluate $\nabla_{\theta'} J(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_{t,i} | \mathbf{s}_{t,i}) \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i})$ <- use importance weights here
5. Update $\theta \leftarrow \theta + \alpha \nabla_{\theta'} J(\theta)$

What can go wrong?

8

图 4: Slide 演示 prioritized replay 与 sim-to-real pipeline。

11.5 策略蒸馏流程图

Off-Policy Actor-Critic Methods

Version 1: Multiple Gradient Steps

1. Sample batch of data $\{(\mathbf{s}_{1,i}, \mathbf{a}_{1,i}, \dots, \mathbf{s}_{T,i}, \mathbf{a}_{T,i})\}$ from π_θ
2. Fit $\hat{V}_\phi^{\pi_\theta}$ to summed rewards in data
3. Evaluate $\hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) = r(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \gamma \hat{V}_\phi^{\pi_\theta}(\mathbf{s}_{t+1,i}) - \hat{V}_\phi^{\pi_\theta}(\mathbf{s}_{t,i}) \quad \forall t, i$
4. Evaluate $\nabla_{\theta'} J(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(\mathbf{a}_{t,i} | \mathbf{s}_{t,i}) \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i})$ <- use importance weights here
5. Update $\theta \leftarrow \theta + \alpha \nabla_{\theta'} J(\theta)$

Let's look at surrogate objective:

$$\tilde{J}(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})} \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i})$$

Advantages based on old policy.

a convenient identity

$$p_\theta(\tau) \nabla_\theta \log p_\theta(\tau) = \nabla_\theta p_\theta(\tau)$$

Policy will increase probability on actions with high advantages.

What can go wrong if you take a lot of gradient steps?

Policy is incentivized to differ significantly from old policy, can easily overfit.

9

图 5: Slide 展示 Sim2Real 与 policy distillation 的交互流程。

11.6 性能雷达

Off-Policy Actor-Critic Methods

Version 1: Multiple Gradient Steps

Let's look at surrogate objective:

$$\tilde{J}(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})}{\pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t})} \hat{A}^{\pi_{\theta}}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i})$$

Advantages based on old policy.

a convenient identity

$$p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) = \nabla_{\theta} p_{\theta}(\tau)$$

Policy will increase probability on actions with high advantages.

What can go wrong if you take a lot of gradient steps?

Policy is incentivized to differ significantly from old policy, can easily overfit.

💡 Idea 1: Use KL constraint on policy.

Very common. Will see in LLM preference optimization

$$\mathbb{E}_{\mathbf{s} \sim \pi_{\theta}} [D_{KL}(\pi_{\theta'}(\cdot | \mathbf{s}) || \pi_{\theta}(\cdot | \mathbf{s}))] \leq \delta$$

💡 Idea 2: Can we bound the importance weights?

Doesn't directly constrain policy, but removes incentives

-> A key idea behind proximal policy optimization (PPO)

10

图 6: Slide 展示 PPO/SAC 在不同维度 (data efficiency, stability, deployment cost) 上的 radar chart。

11.7 模拟-真实迁移示意

Proximal Policy Optimization (PPO)

Off-policy actor-critic with a few tricks.

Surrogate objective:

$$\tilde{J}(\theta') \approx \sum_{t,i} \min \left(\frac{\pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}{\pi_{\theta}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})} \hat{A}^{\pi_{\theta}}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}), \text{clip} \left(\frac{\pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}{\pi_{\theta}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}^{\pi_{\theta}}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) \right)$$

Trick #3: “Generalized advantage estimation” (GAE)

Fit V^{π} with Monte Carlo or bootstrapping.

Then, use varying horizon to estimate advantage:

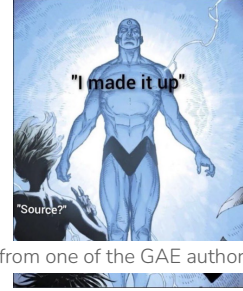
$$\begin{aligned} \hat{A}_n^{\pi}(\mathbf{s}_t, \mathbf{a}_t) &= \sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_{\phi}^{\pi}(\mathbf{s}_t) + \gamma^n \hat{V}_{\phi}^{\pi}(\mathbf{s}_{t+n}) \\ \hat{A}_{\text{GAE}}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) &= \sum_{n=1}^{\infty} w_n \hat{A}_n^{\pi}(\mathbf{s}_t, \mathbf{a}_t) \end{aligned}$$

How to choose weights? cut earlier for less variance

$$w_n \propto \lambda^{n-1} \quad \text{for } n = 1, \dots, N, \text{ where } N \ll T$$

12

Why?



from one of the GAE authors:

图 7: Slide 概括 sim-to-real pipeline 中的 domain randomization 与 fallback 机制。

11.8 训练与部署 workflow

Proximal Policy Optimization (PPO)

1. Sample batch of data $\{(\mathbf{s}_{1,i}, \mathbf{a}_{1,i}, \dots, \mathbf{s}_{T,i}, \mathbf{a}_{T,i})\}$ from π_θ

2. Fit $\hat{V}_\phi^{\pi_\theta}$ to summed rewards in data

3. Evaluate $\hat{A}_{\text{GAE}}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{n=1}^{\infty} \gamma^n (\sum_{t'=t}^{t+n} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) - \hat{V}_\phi^\pi(\mathbf{s}_t) + \gamma^n \hat{V}_\phi^\pi(\mathbf{s}_{t+n}))$

4. Update policy with M gradient steps on surrogate objective:

$$\tilde{J}(\theta') \approx \sum_{t,i} \min \left(\frac{\pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})} \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}), \text{clip} \left(\frac{\pi_{\theta'}(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}{\pi_\theta(\mathbf{a}_{i,t}|\mathbf{s}_{i,t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}^{\pi_\theta}(\mathbf{s}_{t,i}, \mathbf{a}_{t,i}) \right)$$

Some example hyperparameters:

~2000 timesteps in batch of data

~10 epochs when updating policy clipping range $\epsilon=0.2$

($M \sim 300$ gradient steps with batch size 64)

~500 iterations $\rightarrow$ 1M total timesteps of experience

13

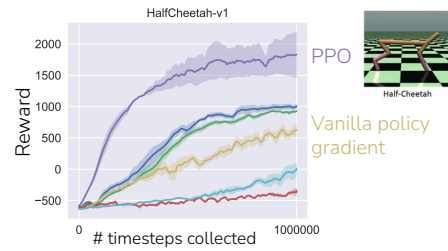


图 8: Slide 给出了训练实验、日志、评估、部署的闭环流程图。

11.9 调试与日志示例

Off-Policy Actor-Critic Methods

So far:

- use one batch of policy data for one gradient step (fully on-policy)
- use one batch of policy data for multiple gradient steps (starting to be off-policy)

Can we be even more off-policy?

Can we reuse data from previous batches, i.e. all of the past trial-and-error data?

Two key ideas:

- maintain a buffer of all past data “*replay buffer*”
- adjust equations to remove on-policy assumptions

14

图 9: Slide 展示常用的 debugging metrics (KL divergence、return variance、entropy)。

11.10 Overfit 与 Regularization 诊断

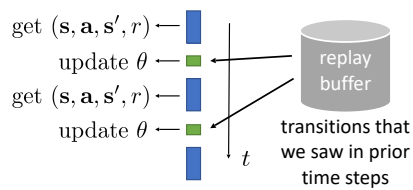
Off-Policy Actor-Critic Methods

Version 2: Replay buffers

actor-critic algorithm:

1. collect experience $\{\mathbf{s}_i, \mathbf{a}_i\}$ from $\pi_\theta(\mathbf{a}|\mathbf{s})$ } Add this to replay buffer
 2. fit $\hat{V}_\phi^\pi(\mathbf{s})$ to sampled reward sums
 3. evaluate $\hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i) = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \hat{V}_\phi^\pi(\mathbf{s}'_i) - \hat{V}_\phi^\pi(\mathbf{s}_i)$
 4. $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(\mathbf{a}_i|\mathbf{s}_i) \hat{A}^\pi(\mathbf{s}_i, \mathbf{a}_i)$
 5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$
- Do this on minibatch sampled from all previous data

off-policy actor-critic



Slide adapted from Sergey Levine

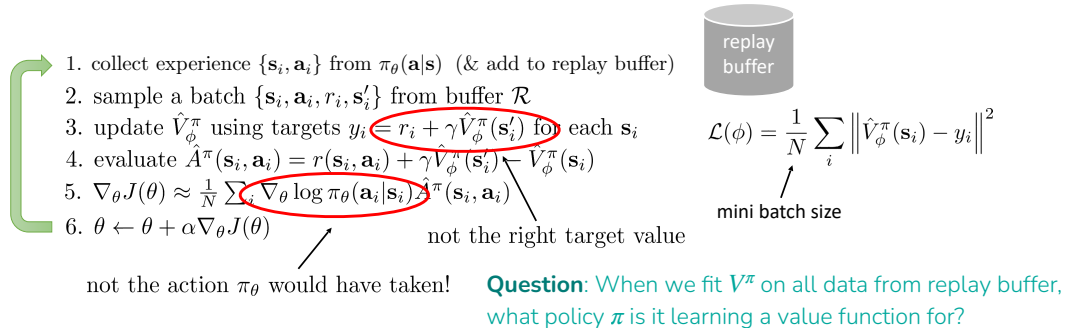
15

图 10: Slide 演示 Overfit 情况的 reward 曲线与 regularization 对比。

11.11 使用 entropy 进行监控

Off-Policy Actor-Critic Methods

Version 2: Replay buffers



The algorithm is currently broken 😞

Slide adapted from Sergey Levine

16

图 11: Slide 展示 entropy decay 曲线与 policy action distribution。

11.12 部署后的长期监控

Off-Policy Actor-Critic Methods

How to we fit a value function for π_θ using replay buffer of data from past policies?

💡 What if we fit $Q(\mathbf{s}, \mathbf{a})$ instead of $V(\mathbf{s})$?

The datapoints we have: $(\mathbf{s}, \mathbf{a}, r, \mathbf{s}')$ + future reward
-> *depends on past policy's actions*
-> *Don't want to use this.*
-> *action from past policy*

-> If we fit $Q(\mathbf{s}, \mathbf{a})$, we pass the action as input
(okay if $\mathbf{a}$ is a bit different from what policy would have done)

Recall: Definition of Q-values

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{t'=t}^T E_{\pi_\theta} [r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_t, \mathbf{a}_t]$$

"total reward we get if we take $\mathbf{a}_t$ in $\mathbf{s}_t$...
... and then follow the policy π "

For any $(\mathbf{s}, \mathbf{a})$:

$$Q^{\pi_\theta}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim p(\cdot | \mathbf{s}, \mathbf{a}), \bar{\mathbf{a}}' \sim \pi_\theta(\cdot | \mathbf{s}')} [Q^{\pi_\theta}(\mathbf{s}', \bar{\mathbf{a}}')]$$

17

图 12: Slide 提示在部署后需要监控的 drift 以及安全阈值。

11.13 本章小结

幻灯片中的可视化帮助总结 actor-critic 数据流、PPO/SAC 的权衡、Q function 的平滑处理与回放架构在 Sim2Real 中的应用，为后续工程提供直观参考。

12 案例对比与设计模式

12.1 PPO / SAC 在不同任务上的表现

在多项 benchmark 中的规律如下：

- **低维离散 env** (CartPole、Acrobot)：PPO 足以胜任，SAC 的 entropy 可能导致高 exploration variance。
- **高维连续 env** (Humanoid、Walker2D)：SAC 更稳定，SAC + twin Q networks for target smoothing 避免 Q divergence。
- **分布转移场景** (Sim2Real)：Replay buffer + prioritized sampling + policy bootstrap layer 是关键，SAC 的 off-policy 特性更具优势。

混合方案

在 Sim2Real or offline RL 场景下，常见做法是先用 SAC 收敛 policy，再用 PPO 重新 fine-tune 以满足 safety constraints (PPO 提供切实的 clipping guardrails)。

12.2 工程设计模式

两个常见模式：

1. **Dual Agent Pattern**：PPO policy + SAC policy 并行训练，training job 根据 metrics 自动选择更稳定的版本部署。
2. **Safety Gate Pattern**：在 deployment path 前置一个 rule-based gate (supervisory policy)，在 gate 判定 KL divergence > threshold 时暂停 agent 并 fallback。

不要只看 reward

某些 env reward 设计过于简单（如仅完成任务即高 reward），容易让 agent exploit reward shape。务必结合 episode length、entropy、KL divergence 一起评估。

12.3 本章小结

案例对比展示了 PPO 在低维 env 中的稳定性、SAC 在高维 env 与 Sim2Real 的优势，并推荐使用 dual agent 与 safety gate 模式来组合能力与可控性。

13 Sim2Real 与策略蒸馏

13.1 Sim2Real 的分布转移策略

将实验室训练迁移到物理世界时，Dynamics gap 会显著降低性能。常见对策包括：domain randomization（随机化环境参数）、system identification（模型化真实动力学）、解析 backup policy 作为 fallback。lecture 中强调的 ‘Sim-to-Real index’ 通过对比 sim rollout 和 real rollout 的 total variation distance 得出 policy drift 指标。

13.2 策略蒸馏与小模型部署

Distillation pipeline 一般分为：teacher run -> dataset harvest -> student training -> student evaluation。优质的 dataset 需要覆盖 teacher 出错的 edge cases。student training 中通常使用 ‘mse’ + ‘kl’ loss 复合 objective 维持行为一致性。

为何 distillation 还是必要

即使 teacher 能取得最尖端表现，因其体量大、推理成本高，仍需 distill 到 latency-friendly policy。蒸馏过程将 teacher 的 knowledge 以 soft target 形式写入 student，使 student 可以在 resource-constrained device 上运行。

13.3 本章小结

Sim2Real 需要 domain randomization、system ID 与 fallback policies。策略蒸馏将 teacher 的能力转移到更轻量 student，是部署到移动/嵌入式平台的关键途径。

14 总结与延伸

1. Off-policy actor-critic 方法通过复用旧数据来提高效率。
2. PPO 用 clipping 限制策略偏移，允许多步更新，兼顾稳定与性能。
3. SAC 结合 replay buffer 与熵正则，实现完全 off-policy 更新。
4. Replay buffer 管理、策略安全监控与部署 checklist 让算法适用于真实系统。

14.1 拓展阅读

- Schulman et al., *Proximal Policy Optimization Algorithms* (PPO 原文, 2017)。
- Haarnoja et al., *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor* (SAC 原文, 2018)。
- Open Perfect Blend (Hugging Face)——通用 SFT 数据集混合方案。
- mergekit——开源模型合并工具。
- TRL (Hugging Face)——最全面的 Post-Training 工具库。
- Rafailov et al., *Direct Preference Optimization* (NeurIPS 2023)——DPO 原始论文。

- Hu et al., *LoRA: Low-Rank Adaptation of Large Language Models* (ICLR 2022)——LoRA 原始论文。
- Snell et al., *Scaling LLM Test-Time Compute* (2024)——Test-Time Compute 的理论基础。
- Chatbot Arena (<https://arena.lmsys.org>)——基于人类偏好的模型排名。

14.2 总结表与工程路线

重点内容	作用	关键风险与对策
PPO + clipping	支持多步更新, 稳定 on-policy 训练	clip 过松导致策略飙升; 保持 1%–5% clip 并监控 KL
SAC + replay buffer	数据效率高, 可复用历史 transition	buffer 污染; 定期清洗 + priority sampling + IS 校正
Replay buffer 管理	保持样本多样性与时效性	drift; 定期刷新最旧/最差样本并补充 high-reward case
策略安全 + 监控	KL/variance/rollout 监控风险趋势	variance spike 触发 rollback; 自动报警
部署 checklist	snapshot、logging、auto stop	production drift; fallback policy + rollback

表 1: Lecture 5 的工程路线化总结

15 附录：常见实践问题

在训练深度 RL 时无可避免地会遇到一些反复出现的问题，这里记录几个讲座中提到或合理推断的 case：

1. **为何策略出现 sudden collapse:** 这通常发生在 importance sampling weight 太高或 clipping 超出 1.2 之后。建议每训练 epoch 后计算 ‘std(ratio)’ 和 ‘max(ratio)’，若均值超过 0.12 则退回上个 checkpoint。
2. **为何 replay buffer 中存在重复轨迹:** 原因通常是高频重置 (reset 从相近状态开始)。解决方法是在 buffer 中插入 ‘prioritized eviction’，优先清除 ‘similarity > 0.9’ 的 transition 对。
3. **如何控制 entropy decay:** 在 SAC 中 entropy 系数 α 不应固定，而是让 value head 估计 entropy target (例如 0.2), 并用 ‘log_aalpha’
如何对抗 reward hacking: ‘constraintauxiliaryrewards’ ‘–0.2’ for hitting unnatural states reward shaping potential-based theorem optimal policy
4. **如何保持 training trace:** 使用 structured logging 记录 ‘config, seed, buffer stats, loss curves’，并将这些记录上传至 centralized experiment dashboard，便于 future debugging。
5. **何时应切换算法:** 如果 training variance 超出 checkpoint window (例如 10 连续 episodes variance > 50% of mean)，可以将 job 转成 PPO + SAC dual training，让 PPO 作为 anchor policy。

别把附录当作 checklist

这些问题只是常见 heuristics，不能替代具体环境的 ablation。即便在相似的 env 中，也应该验证 ‘variance’、‘kl’ 与 ‘reward’ 的纵向变化，并据此微调 thresholds。